

```
!pip install skl2onnx onnxruntime
```

```
Collecting skl2onnx
  Downloading skl2onnx-1.20.0-py3-none-any.whl.metadata (4.0 kB)
Collecting onnxruntime
  Downloading onnxruntime-1.25.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_2
Collecting onnx>=1.2.1 (from skl2onnx)
  Downloading onnx-1.21.0-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64
Requirement already satisfied: scikit-learn>=1.1 in /usr/local/lib/python3.12/di
Collecting flatbuffers (from onnxruntime)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Requirement already satisfied: numpy>=1.21.6 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: typing_extensions>=4.7.1 in /usr/local/lib/python
Requirement already satisfied: ml_dtypes>=0.5.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12
Downloading skl2onnx-1.20.0-py3-none-any.whl (317 kB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 317.2/317.2 kB 9.3 MB/s eta 0:00:00
Downloading onnxruntime-1.25.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.0/18.0 MB 155.3 MB/s eta 0:00:00
Downloading onnx-1.21.0-cp312-abi3-manylinux_2_27_x86_64.manylinux_2_28_x86_64.w
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.6/17.6 MB 131.2 MB/s eta 0:00:00
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Installing collected packages: flatbuffers, onnxruntime, onnx, skl2onnx
Successfully installed flatbuffers-25.12.19 onnx-1.21.0 onnxruntime-1.25.1 skl2o
```

```
!pip install lxml
```

```
Collecting lxml
  Downloading lxml-6.1.0-cp312-cp312-manylinux_2_26_x86_64.manylinux_2_28_x86_64
Downloading lxml-6.1.0-cp312-cp312-manylinux_2_26_x86_64.manylinux_2_28_x86_64.w
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.2/5.2 MB 89.2 MB/s eta 0:00:00
Installing collected packages: lxml
Successfully installed lxml-6.1.0
```

```
!pip install thefuzz[speedup]
```

```
Collecting thefuzz[speedup]
  Downloading thefuzz-0.22.1-py3-none-any.whl.metadata (3.9 kB)
Collecting rapidfuzz<4.0.0,>=3.0.0 (from thefuzz[speedup])
  Downloading rapidfuzz-3.14.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_
Downloading rapidfuzz-3.14.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x8
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.1/3.1 MB 3.9 MB/s eta 0:00:00
Downloading thefuzz-0.22.1-py3-none-any.whl (8.2 kB)
Installing collected packages: rapidfuzz, thefuzz
Successfully installed rapidfuzz-3.14.5 thefuzz-0.22.1
```

```

import pandas as pd

file_names = [
    'cleaned_2017-18.csv', 'cleaned_2018-19.csv',
    'cleaned_2020-21.csv', 'cleaned_2021-22.csv',
    'cleaned_2023-24.csv'
]

dataframes = []
for file in file_names:
    df_temp = pd.read_csv(file)
    dataframes.append(df_temp)

master_df = pd.concat(dataframes, ignore_index=True)

master_df = master_df.rename(columns={'Avg Mins per game': 'Total_Minutes'})

master_df = master_df[master_df['Total_Minutes'] > 0]

master_df['Key_Passes_p90'] = (master_df['Key passes p90'] / master_df['Total_Minutes']) * 90
master_df['Interceptions_p90'] = (master_df['Interceptions p90'] / master_df['Total_Minutes']) * 90
master_df['Tackles_Won_p90'] = (master_df['Tackles Won p90'] / master_df['Total_Minutes']) * 90
master_df['Prog_Passes_p90'] = (master_df['Progressive passes p90'] / master_df['Total_Minutes']) * 90
master_df['Prog_Carries_p90'] = (master_df['Progressive carries p90'] / master_df['Total_Minutes']) * 90

features_to_keep = [
    'player', 'pos', 'season', 'Total_Minutes',
    'Goals', 'Assists',
    'Goals p 90', 'Assists p 90', 'Expected Goals',
    'Shot creating actions p 90', 'Key_Passes_p90',
    'Interceptions_p90', 'Tackles_Won_p90',
    'Prog_Passes_p90', 'Prog_Carries_p90'
]

training_data = master_df[features_to_keep]

print(" Data successfully merged and cleaned ")
print(f"Total historical player seasons available to learn from: {len(training_data)}")
print(training_data.head(3))

```

```

Data successfully merged and cleaned
Total historical player seasons available to learn from: 7852

```

	player	pos	season	Total_Minutes	Goals	Assists	\
0	Patrick van Aanholt	DF	2017-2018	2184	5	1	
7	David Abraham	DF	2017-2018	2302	0	2	
8	Tammy Abraham	FW	2017-2018	1726	5	1	

	Goals p 90	Assists p 90	Expected Goals	Shot creating actions p 90	\
0	0.21	0.04	3.1	1.90	
7	0.00	0.08	0.4	0.78	
8	0.26	0.05	6.8	2.19	

	Key_Passes_p90	Interceptions_p90	Tackles_Won_p90	Prog_Passes_p90	\
0	0.741758	1.936813	1.318681	3.791209	
7	0.273675	1.563858	1.251086	4.026933	
8	0.625724	0.052144	0.417149	1.042874	

	Prog_Carries_p90
0	1.895604
7	0.664639
8	0.990730

```

import pandas as pd
import re
from thefuzz import process, fuzz
import requests

print("Starting Wikipedia Scraper")

award_to_season = {
    2023: '2022-2023',
    2022: '2021-2022',
    2021: '2020-2021',
    2019: '2018-2019',
    2018: '2017-2018'
}

wiki_dataframes = []

headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
}

for year, season in award_to_season.items():
    url = f'https://en.wikipedia.org/wiki/{year}_Ballon_d%270r'
    print(f"Fetching {year} points...")

    try:

        response = requests.get(url, headers=headers)
        response.raise_for_status()
        tables = pd.read_html(response.text)
        voting_table = None

        for table in tables:
            cols = [str(c).lower() for c in table.columns]
            if any('player' in c for c in cols) and any(('points' in c or 'pts'

```

```

        voting_table = table.copy()
        break

    if voting_table is not None:
        col_map = {}
        for col in voting_table.columns:
            if 'player' in str(col).lower(): col_map[col] = 'Wiki_Name'
            elif 'points' in str(col).lower() or 'pts' in str(col).lower():

        voting_table = voting_table.rename(columns=col_map)
        voting_table = voting_table[['Wiki_Name', 'Ballon_Points']].dropna()

        voting_table['Ballon_Points'] = voting_table['Ballon_Points'].astype(int)

        voting_table['season'] = season
        wiki_dataframes.append(voting_table)
    else:
        print(f"No voting table found for {year} at {url}")

except requests.exceptions.RequestException as e:
    print(f"Could not fetch {year}: {e}")
except ValueError as e:
    print(f"Error processing HTML for {year}: {e}")
except Exception as e:
    print(f"An unexpected error occurred for {year}: {e}")

if not wiki_dataframes:
    print("No Wikipedia data was successfully fetched. Cannot proceed with merge")

else:
    all_wiki_points = pd.concat(wiki_dataframes, ignore_index=True)

    print("\n Running Fuzzy Match to the Master CSV Data ")

    final_merged_data = []

    for season in award_to_season.values():
        df_csv_season = training_data[training_data['season'] == season].copy()
        csv_names = df_csv_season['player'].tolist()

        df_wiki_season = all_wiki_points[all_wiki_points['season'] == season].copy()

        def get_csv_name(wiki_name):
            match, score = process.extractOne(wiki_name, csv_names, scorer=fuzz.partial_ratio)
            if score >= 75:
                return match
            return None

        df_wiki_season['Matched_Player'] = df_wiki_season['Wiki_Name'].apply(get_csv_name)

```

```

merged_season = pd.merge(
    df_csv_season,
    df_wiki_season[['Matched_Player', 'Ballon_Points']],
    left_on='player',
    right_on='Matched_Player',
    how='left'
)
final_merged_data.append(merged_season)

final_ml_dataset = pd.concat(final_merged_data, ignore_index=True)

final_ml_dataset['Ballon_Points'] = final_ml_dataset['Ballon_Points'].fillna(0)

final_ml_dataset = final_ml_dataset.drop(columns=['Matched_Player'])

print("\n done! final dataset ready for training")

print(final_ml_dataset.sort_values(by='Ballon_Points', ascending=False)[['p

```

Starting Wikipedia Scraper

Fetching 2023 points...

/tmp/ipykernel_970/726641404.py:31: FutureWarning: Passing literal html to 'read_html' is deprecated.
 tables = pd.read_html(response.text)

Fetching 2022 points...

/tmp/ipykernel_970/726641404.py:31: FutureWarning: Passing literal html to 'read_html' is deprecated.
 tables = pd.read_html(response.text)

Fetching 2021 points...

/tmp/ipykernel_970/726641404.py:31: FutureWarning: Passing literal html to 'read_html' is deprecated.
 tables = pd.read_html(response.text)

Fetching 2019 points...

/tmp/ipykernel_970/726641404.py:31: FutureWarning: Passing literal html to 'read_html' is deprecated.
 tables = pd.read_html(response.text)

Fetching 2018 points...

Running Fuzzy Match to the Master CSV Data

done! final dataset ready for training

	player	season	Ballon_Points
5274	Luka Modrić	2017-2018	753.0
4111	Lionel Messi	2018-2019	686.0
4474	Virgil van Dijk	2018-2019	679.0
2964	Lionel Messi	2020-2021	613.0
2853	Robert Lewandowski	2020-2021	580.0
1229	Karim Benzema	2021-2022	549.0
4318	Cristiano Ronaldo	2018-2019	476.0
5461	Cristiano Ronaldo	2017-2018	476.0
697	Lionel Messi	2022-2023	462.0
2762	Jorginho	2020-2021	460.0

/tmp/ipykernel_970/726641404.py:31: FutureWarning: Passing literal html to 'read_html' is deprecated.
 tables = pd.read_html(response.text)

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestRegressor
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType, StringTensorType
import numpy as np
from google.colab import files

print("STEP 4: preparing the machine learning process")

numeric_features = [
    'Total_Minutes', 'Goals', 'Assists', 'Goals p 90', 'Assists p 90',
    'Expected Goals', 'Shot creating actions p 90', 'Key_Passes_p90',
    'Interceptions_p90', 'Tackles_Won_p90',
    'Prog_Passes_p90', 'Prog_Carries_p90'
]
categorical_features = ['pos']

ml_data_clean = final_ml_dataset.dropna(subset=numeric_features + categorical_f

X = ml_data_clean[numeric_features + categorical_features]
y = ml_data_clean['Ballon_Points']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=300, max_depth=10, random_
])

sample_weights = np.where(y > 0, 10.0, 1.0)

print("\n STEP 5: training the model")
model_pipeline.fit(X, y, regressor__sample_weight=sample_weights)

print("\n STEP 6: model rank for 2022-2023 season")
# Grab just the players from the 22-23 season
season_22_23 = ml_data_clean[ml_data_clean['season'] == '2022-2023'].copy()
X_season = season_22_23[numeric_features + categorical_features]
```

```

season_22_23['AI_Predicted_Points'] = model_pipeline.predict(X_season)

top_15_ai = season_22_23.sort_values(by='AI_Predicted_Points', ascending=False)

print("\n🏆 AI PREDICTED BALLON D'OR TOP 15 (2022-2023) 🏆")
print(top_15_ai[['player', 'pos', 'AI_Predicted_Points', 'Ballon_Points']].to_s

print("\n--- STEP 7: EXPORTING TO ONNX ---")

initial_types = []
for feature_name in numeric_features:
    initial_types.append((feature_name, FloatTensorType([None, 1])))
for feature_name in categorical_features:
    initial_types.append((feature_name, StringTensorType([None, 1])))

onnx_model = convert_sklearn(model_pipeline, initial_types=initial_types)
onnx_filename = "poty_model.onnx"

with open(onnx_filename, "wb") as f:
    f.write(onnx_model.SerializeToString())

print(f"\n successful! Model saved as '{onnx_filename}'.")
files.download(onnx_filename)

```

STEP 4: preparing the machine learning process

STEP 5: training the model

STEP 6: model rank for 2022-2023 season

🏆 AI PREDICTED BALLON D'OR TOP 15 (2022-2023) 🏆

	player	pos	AI_Predicted_Points	Ballon_Points
	Lionel Messi	FW	317.732015	462.0
	Erling Haaland	FW	303.751667	357.0
	Kylian Mbappé	FW	252.548466	270.0
	Kevin De Bruyne	MF	77.268777	100.0
	Harry Kane	FW	66.237051	4.0
	Konstantinos Mavropanos	DF	42.348565	0.0
	Dani Ceballos	MF	40.412419	0.0
	Antoine Griezmann	FW	33.685952	4.0
	Mohamed Salah	FW	31.217000	13.0
	Vinicius Júnior	FW	30.719872	49.0
	Rodri	MF	28.321916	57.0
	Karim Benzema	FW	24.018059	6.0
	Florent Ogier	DF	19.803097	0.0
	Robert Lewandowski	FW	19.173841	12.0
	Rodri	FW	18.227991	57.0

--- STEP 7: EXPORTING TO ONNX ---

successful! Model saved as 'poty_model.onnx'.

